

university of groningen

# Performance Engineering: Lecture #13

## WMCS045-05

**Dirk Pleiter**

Bernoulli Institute

2026-10-14

# Overview



1. Network Architectures
2. MPI Introduction
3. First MPI Program
4. Blocking Communication
5. Final

# Content



1. Network Architectures
2. MPI Introduction
3. First MPI Program
4. Blocking Communication
5. Final

# Network Architectures: Introduction



- Key components relevant for communication performance
  - Link technology and speed
  - Link communication protocols
    - Protocol for communication of messages between nodes
  - Communication library protocol
    - Protocol for interacting with the communication library
  - Network topology

# Message Communication Protocols




- **Eager protocol**:
  - Sender pushes message regardless of receiver state
  - Low overhead
  - Useful for small messages
- **Rendezvous protocol**:
  - Handshake happens between the sender and the receiver before sending data
  - High overhead due handshake involving control messages
  - Useful for large messages

# Blocking versus Non-blocking Communication



- **Blocking call** = Call only returns once request has been served
  - Caller will wait, potentially missing the opportunity to do useful work
- **Non-blocking call** = Call returns before request has been served
  - Caller will have to check for completion

# One- Versus Two-sided Communication



- **One-sided communication** = Sender (or receiver) pushes (or pulls) a message without active involvement of the receiver (or sender)
  - Get/put semantics
  - **Advantages(s)**
    - Typically lower overhead
  - **Disadvantage(s)**
    - Need for explicit synchronization, i.e. more complexity
- **Two-sided communication** = Sender and receiver cooperate for communicating a message
  - Send/receive semantics
  - **Advantages(s)**
    - More intuitive communication model
  - **Disadvantage(s)**
    - Higher latency, more overhead

# Remote Direct Memory Access (RDMA)



- **Idea**: Allow one process to directly access the (private) memory space of another process running on another node without involving the CPU cores or operating system of the other node
- **Implementation** of an **RDMA "put"** on a NIC
  - Initial setup
  - For each communication:
    - Sender CPU places request into a queue
    - Sender NIC fetches request
    - Sender NIC gets data from local memory
    - Sender NIC sends message including destination addresses
    - Receiver NIC receives data and puts it into local memory
- Network technologies supporting RDMA
  - Infiniband
  - Ethernet supporting RoCE (RDMA over Converged Ethernet)

# Network Topology



- **Network topology** refers to the way in which the network of computing nodes is connected
- **Physical topology** = Topology of the physical network
- **Logical topology** = Logical view on the way network is connected
- Simplest topology: point-to-point topology
  - Permanent link between two **endpoints**

# Network Topology Evaluation Metric



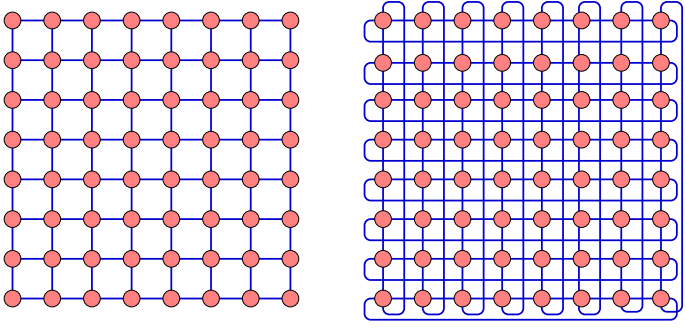
- Diameter
  - Maximum minimal distance between 2 nodes
  - Smaller is better
- Connectivity
  - The minimum number of arcs that must be removed to break it into two disconnected networks
  - Larger is better
- Bi-section width
  - The minimum number of arcs that must be removed to partition the network into two equal halves
  - Larger is better
- Bi-section bandwidth
  - Aggregate bandwidth of all links connecting two equal halves
  - Larger is better
- Costs
  - Smaller is better

# Cartesian Network Topologies (1/2)

- 1-dimensional linear array and ring:



- 2-dimensional mesh and torus:



# Cartesian Network Topologies (2/2)



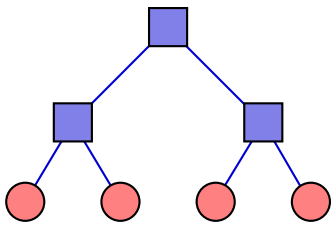
- Evaluation:

| | Mesh | Torus |
|---|---|---|
| Diameter | $d(P^{1/d} - 1)$ | $d(P^{1/d}/2)$ |
| Connectivity | $d$ | $2d$ |
| Bi-section width | $P^{(d-1)/d}$ | $2P^{(d-1)/d}$ |

# Network Topology: Tree

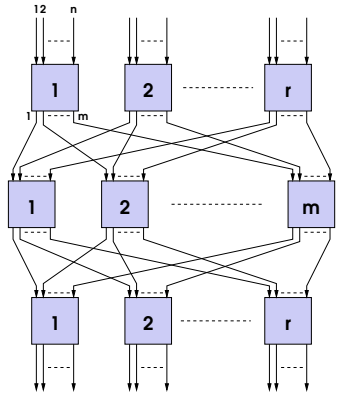


- Switch-based network
- Evaluation binary tree:
  - Diameter: $O(\log(P))$
  - Connectivity: $1$
  - Bi-section width: $1$
  - Costs: $O(P)$
- Number of links between upper levels smaller
  ☞ **Blocking network topology**



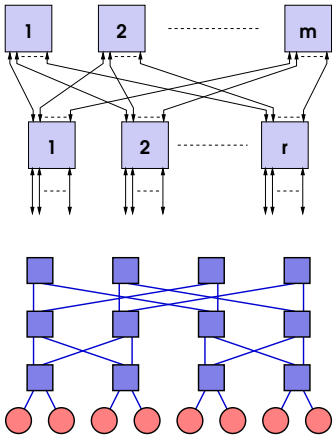
# Network Topology: Clos Network

- $N$-stage switch-based network
  - Here: $N = 3$
- Network is characterised by
  - $r$: number of ingress stage switches
  - $n$: number of ingress ports at ingress stage
  - $m$: number of egress ports at ingress stage
- Evaluation:
  - Diameter: $N + 1$
  - Connectivity: $m$ (at switch-level)
  - Bi-section width: $r\frac{m}{2}$
- A Clos network can be made **non-blocking**
  - Non-blocking for $m \geq n$ if one allows for rearrangements
  - Strictly non-blocking if $m \geq 2n - 1$



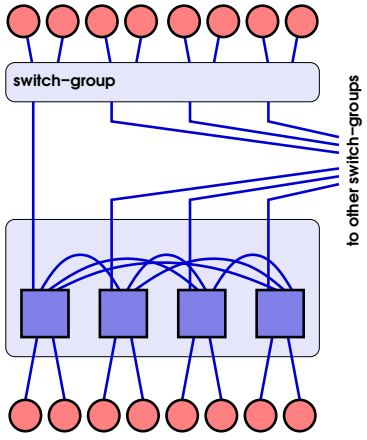
# Network Topology: Folded Clos Network

- Topology obtained by folding ingress and egress stage
  - Example: folded 3-stage Clos
- Evaluation of Clos and Folded Clos network:
  - Diameter $= N + 1$
  - Connectivity $= m$
  - Bi-section width $= r\frac{m}{2}$
- Closely related to **fat-tree** topology
  - Here: similar to folded 5-stage Clos



# Network Topology: Dragonfly

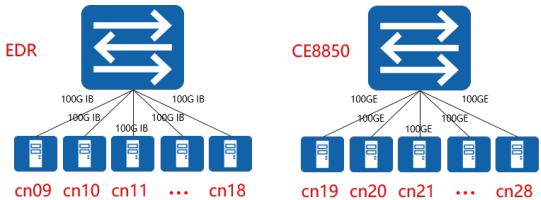
- Multi-layer network
  - Switch layer
  - Switch-group layer
  - System layer
- All-to-all connectivity within layer
- Network parameters
  - $p$: number of nodes per switch
  - $a$: number of switches per group
  - $h$: number of links per switch to other groups
- Evaluation:
  - Diameter: $5$
  - Connectivity: $ah - b$ (at system layer; $b$ = bundel width)
  - Bi-section width: $\left(\frac{a\,h-1}{2}\right)^2$



# Example: HAICGU

- Fat-tree topology
- Two network technologies supported
  - EDR Infiniband (Mellanox MSB7890-ES2F)
  - 100GE (Huawei CloudEngine 8850)
- All nodes equipped with Mellanox ConnectX-5 adapters
- Select the Slurm partitions `cn-ib*` or `cn-eth`

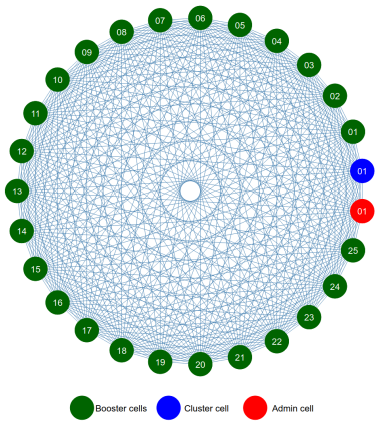
[L. Pichetti et al.; 2024]



# Example: JUPITER (1/2)

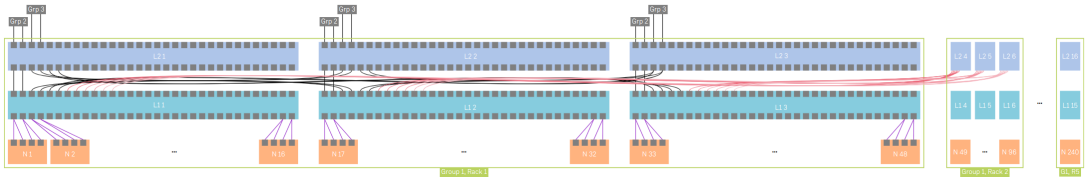
- JUPITER is the first exascale system in Europe
- Installation site: Jülich Supercomputing Centre
- Number of nodes:
  - ~6,000 GPU-accelerated nodes
  - ~1,300 CPU-only nodes
- Network topology: **Dragonfly+**
  - Fat-tree within a switch-group
  - All-to-all between switch-groups

[A. Herten; 2025]



# Example: Jupiter (2/2)

[A. Herten; 2025]




- **Key benefit of dragonfly**: Moderate number of long-distance optical cables to interconnect switch-groups
  - Cables have to extend multiple containers

# Content



1. Network Architectures
2. MPI Introduction
3. First MPI Program
4. Blocking Communication
5. Final

# MPI Introduction



- **MPI** = Message-Passing library Interface specification
  - Model: Data is moved from the address space of one process to that of another process through cooperative operations on each process
- Specification defined by MPI Forum
  - Members of forum: academic and commercial institutions
  - History:

| | |
|---|---|
| May 1994: | Release of first version MPI-1.0 |
| September 2009: | Release of version MPI-2.2 |
| June 2015: | Release of version MPI-3.1 |
| November 2023: | Release of version MPI-4.1 |
| June 2025: | Release of version MPI-5.0 |

# MPI Implementations



- MPI is only a **specification**, not an **implementation**
- Most popular implementations
  - OpenMPI (U Tennesse, LANL, Indiana U plus collaborators)
  - MPICH (ANL plus collaborators)
  - MVAPICH (Ohio State U plus collaborators)
- Different implementations have different advantages/disadvantages. Performance may differ on different platforms and depend on application.

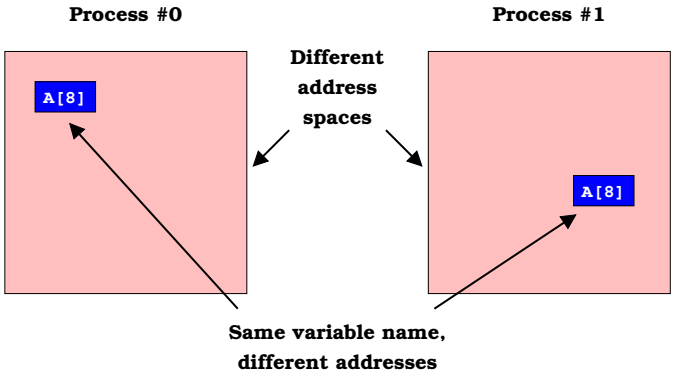
# Message Passing Model (1/2)



- MPI-based parallel programs consist of a **set of separate processes**
  - A process is an active instance of a program with its own program counter and address space
  - The processes can run on the same node or on different nodes
- Typically, **all processes execute the same program**, but operate on different data
  - **SPMD** = Single Program Multiple Data
- Data has to be communicated explicitly between processes
  - Different communication patterns are supported:
    - Point-to-point communication operations
    - Collective communication operations
- Programmer has to take care of data distribution, communication and synchronisation

# Message Passing Model (2/2)

Beware: Variables of the same program with the same name in different processes are different variables stored at different memory addresses



# Why Using MPI? (1/2)



- **Observation**: MPI has become a standard parallel programming model and is supported on essentially all HPC systems
  - MPI replaced various other message passing solutions
- **Portability**: An application developed in a standard compliant manner on one system will run also on other systems that support an MPI implementation
  - In practice, some system specific optimizations may be required
- **Functionality**: Increasingly rich set of features
- **Wide uptake**: MPI is used for the vast majority of parallel applications leading to
  - Many training opportunities and widely available expertise
  - Broad eco-system

# Why Using MPI? (2/2)



- **Availability**: MPI runs on all Linux and most other systems
  - Open source implementations are available and distributed with popular Linux distributions
- **Performance opportunities**: MPI is co-developed with HPC solution providers that perform significant efforts optimizing the software
  - Network technology providers provide optimized back-ends
  - HPC system providers provide there own implementations of MPI, e.g. Cray MPI, IBM Spectrum MPI

# Content



1. Network Architectures
2. MPI Introduction
3. First MPI Program
4. Blocking Communication
5. Final

# First MPI Program: Hello World in C



```c
#include <stdio.h>
#include <mpi.h>

int main (int argc, char *argv[])
{
  int irank, isize;

  MPI_Init(&argc, &argv);

  MPI_Comm_rank(MPI_COMM_WORLD, &irank);
  MPI_Comm_size(MPI_COMM_WORLD, &isize);

  printf("Processor %d of %d: Hello World!\n", irank, isize);

  MPI_Finalize();

  return 0;
}
```

# First MPI Program: Comments



- Need to include MPI header files or declare use of MPI module to make MPI functions, variable types and constants declared or defined
  - C or C++: `#include <mpi.h>`
  - Fortran 2008 or later: `USE mpi_f08`
  - Earlier Fortran: `USE mpi` or `INCLUDE 'mpif.h'`
- `MPI_Init` must be the first MPI function that is called (few exceptions)
- `MPI_Finalize` is required to clean-up all MPI states
- C-language binding
  - MPI used through function calls
  - All functions return an error code of type `int`

# MPI Initialisation



- Single-threaded case

```
int MPI_Init(int *argc, char ***argv)
```

- Instead of the pointers `argc` and `argv` one may also provide `NULL`

```
int MPI_Init(NULL, NULL)
```

- Multi-threaded case

```
int MPI_Init_thread(int *argc, char ***argv,
                    int required, int *provided)
```

  with `required` taking the following values
  - `MPI_THREAD_SINGLE`: Only one thread will execute
  - `MPI_THREAD_FUNNELED`: Only main thread makes MPI calls
  - `MPI_THREAD_SERIALIZED`: Only one threat at a time makes MPI calls
  - `MPI_THREAD_MULTIPLE`: No restrictions

# MPI Finalisation



- Routine to clean up all MPI state

```
int MPI_Finalize(void)
```

- Before calling for finalisation, a process must perform all MPI calls to complete its involvement in MPI communications

# MPI Communicators



- MPI **communicators** (more precisely: intra-communicators) allow to group MPI processes
  - Processes within a communicator can communicate among each other
- MPI communicators define a communication context
  - Communications within different communicators will never be mixed up
- Predefined Communicators
  - `MPI_COMM_WORLD`: Group including all processes
  - `MPI_COMM_SELF`: Group containing only the current process

# MPI Process Identification: Rank



- All processes within a communicator are assigned a unique identify called **rank**
  - Ranks are integer values starting from zero
- An MPI process may be part of multiple communicators and therefore have multiple ranks
  - Typically, rank refers to the identity of a process within the communicator `MPI_COMM_WORLD`

# Compiling MPI Programs



- MPI programs should be compiled using the wrapper compilers provided by the different MPI implementations, e.g.
  - C:
    - mpicc, mpiicc
  - C++:
    - mpicxx, mpiCC, mpic++, mpiicpc
- Most wrappers allow to show what they are doing
  - Example OpenMPI: `mpicc --showme`
- May need to load modules, e.g. on HAICGU: `ml GCC OpenMPI`

# Running MPI Programs



- Starting an MPI program requires a tool called `mpiexec`
  - Not standard compliant alternative: `mpirun`
- Typically, a single MPI is executed:

  ```
  mpiexec -n 8 ./hello_mpi.x
  ```

  - The option `-n` <numprocs> defines the number of processes that are started
  - Other options are MPI implementation specific
- MPI also allows to execute multiple programs, e.g.

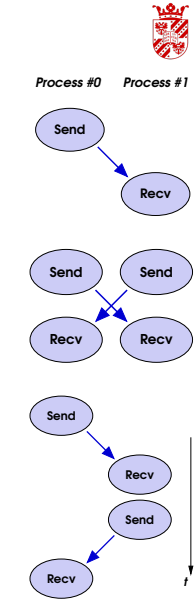
  ```
  mpiexec -n 1 ./master.x : -n 23 ./worker.x
  ```

# Content



1. Network Architectures
2. MPI Introduction
3. First MPI Program
4. Blocking Communication
5. Final

# Digression: Communication Patterns




- Communication patterns involving a pair of processes
- **Ping**:
  - Process #0 sends a message to process #1 (Ping)
- **PingPing**:
  - Each process sends a message to the other process
- **PingPong**:
  - Process #0 sends a message to process #1 (Ping)
  - After receiving the message process #1 returns it to process #0 (Pong)

# Ping in C using MPI



```c
#include <stdio.h>
#include <mpi.h>

int main(int argc ,char *argv[])
{
  int irank;
  int x, y;
  int src, dst;
  MPI_Status status;

  MPI_Init(&argc, &argv);
  MPI_Comm_rank(MPI_COMM_WORLD, &irank);

  x = 100 + irank; y = -1;
  src = 1; dst = 0;

  if (irank == src) {
    MPI_Send(&x, 1, MPI_INT, dst, 99,
             MPI_COMM_WORLD);
    printf("#%d: have sent x=%d\n",
           irank, x);
  }
  else if (irank == dst) {
    MPI_Recv(&y, 1, MPI_INT, src, 99,
             MPI_COMM_WORLD, &status);
    printf("#%d: have received y=%d\n",
           irank, y);
  }

  MPI_Finalize();

  return 0;
}
```

# Standard `MPI_Send`



- Syntax of a blocking send operation:

```
int MPI_Send(
  const void* buf,         /* Pointer to send buffer */
  int count,               /* Number of elements     */
  MPI_Datatype datatype,   /* Data type              */
  int dest,                /* Rank of destination    */
  int tag,                 /* Message tag            */
  MPI_Comm comm            /* Communicator           */
);
```

- The **message buffer** is described by `buf`, `count` and `datatype`
- The **target** is identified by `dest` and `comm`
- Messages are sent with an accompanying user-defined integer **tag** that is used by receiver for identification
  - Allow send-receiver pairs to perform concurrent communications

# MPI Data Types



- MPI is aware of intrinsic data types in the supported languages (C, Fortran, C++)
- Benefits
  - No need for buffer length in Bytes to be computed by programmer
  - Facilitate some support of type checking
  - Automatic data conversion, e.g.
    - Little versus big endian
    - Interfacing C and Fortran routines

# MPI Predefined Data Types



| MPI data type | C data type |
|---|---|
| `MPI_CHAR` | `char` |
| `MPI_INT` | `signed int` |
| `MPI_FLOAT` | `float` |
| `MPI_DOUBLE` | `double` |

# Standard `MPI_Recv`



- Syntax of a blocking receive operation:

```
1 int MPI_Receive(
2   const void* buf,        /* Pointer to receive buffer */
3   int count,              /* Number of elements        */
4   MPI_Datatype datatype, /* Data type                 */
5   int source,             /* Rank of source            */
6   int tag,                /* Message tag               */
7   MPI_Comm comm,          /* Communicator              */
8   MPI_Status *status      /* Status object             */
9 );
```

- Waits until a matching (on source and tag) message is received
  - After function returned, receive buffer can be used
- Alternative to a specific source or tag: `MPI_SOURCE_ANY` or `MPI_TAG_ANY`
- The `status` object on return contains further information
- Receiving fewer than `count` occurrences of `datatype` is OK, but receiving more is an error

# Return Status Object



- Returned information
  - `MPI_SOURCE`: Source of received message
  - `MPI_TAG`: Tag of received message
  - `MPI_ERROR`: Error code
- Language binding
  - C: Structure with 3 elements, e.g. `status.MPI_SOURCE`

# Error Codes



- The MPI specification leaves it to the implementations to define specific error codes, which must comply to the following rules
  - Predefined code $0$ $=$ `MPI_SUCCESS`
  - Other codes must be in the range $[1, \texttt{MPI_ERR_LASTCODE}]$
- The standard defines a set of error classes and a function to translate error code to error class:

```
int MPI_Error_class(int errorcode, int *errorclass)
```

- Error class examples:

| | |
|---|---|
| `MPI_ERR_BUFFER` | Invalid buffer pointer |
| `MPI_ERR_RANK` | Invalid rank |

# Standard `MPI_Sendrecv`



- If all processes send and receive messages (PingPong pattern) send and receive can be initiated using a single MPI call:

```
int MPI_Sendrecv(
  const void *sendbuf,    /* Send buffer          */
  int sendcount,          /* Number of elements   */
  MPI_Datatype sendtype,  /* Data type            */
  int dest,               /* Rank of destination  */
  int sendtag,            /* Send tag             */
  void *recvbuf,          /* Receive buffer       */
  int recvcount,          /* Number of elements   */
  MPI_Datatype recvtype,  /* Data type            */
  int source,             /* Rank of source       */
  int recvtag,            /* Receive tag          */
  MPI_Comm comm,          /* Communicator         */
  MPI_Status *status      /* Status object        */
);
```

# Timing Measurement



- MPI defines a timer that returns time in units of seconds:

  ```
  double MPI_Wtime(void)
  ```

- Example program:

```
1 double tstart = MPI_Wtime();
2 /* stuff to be timed */
3 double tend = MPI_Wtime();
4 printf("Execution time: %.2e seconds\n",tend-tstart);
```

- Time is measured as difference to some time in the past that is guaranteed not to change during process execution
- The times returned are local to the node that called them
- Time resolution can be checked using `MPI_Wtick()`

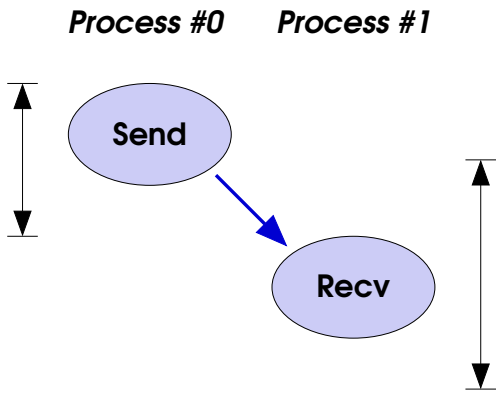
# Time Measurement: Pitfall (1/3)



- Will the modified version of the MPI Ping program correctly measure the time to communicate one integer?

```
1   src = 1; dst = 0;
2
3   double tstart = MPI_Wtime();
4   if (id == src) {
5     MPI_Send(&x, 1, MPI_INT, dst, 99, MPI_COMM_WORLD);
6     printf("#%d: have sent x=%d\n", id, x);
7   }
8   else if (id == dst) {
9     MPI_Recv(&y, 1, MPI_INT, src, 99, MPI_COMM_WORLD, &status);
10    printf("#%d: have received y=%d\n", id, y);
11  }
12  double tend = MPI_Wtime();
13  printf("Time for communication: %.2es\n", tend - tstart);
```

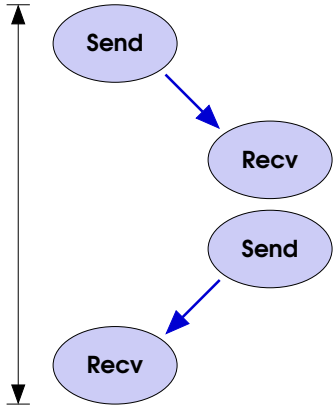
# Time Measurement: Pitfall (2/3)




- **Answer**: No
- **Process #0**: measures the time it takes to complete the send operation (plus printing a string)
- **Process #1**: measures the time from an arbitrary starting point until the receive operation completed

# Time Measurement: Pitfall (3/3)

- **Alternative**: Implement a PingPong communication pattern to measure **round-trip time**



# Content



1. Network Architectures
2. MPI Introduction
3. First MPI Program
4. Blocking Communication
5. Final

# Summary Lecture #13



- Network architecture
  - Link and communication library protocols
  - Network topologies
- MPI
  - Basics
  - Blocking point-to-point communication

# Questions?



[Wiki Media]